

A PROJECT REPORT ON

WOMEN SAFETY ALERT AND LIVE TRACKING SYSTEM

Domain: IoT / Mobile Application

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 IoT Hardware Components	
	4.3 Software Specification	
V	SYSTEM ANALYSIS	10

	5.1 Feasibility Study	
	5.2 About the Software	
VI	SYSTEM DESIGN	13
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	22
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	30
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	31

CHAPTER I

INTRODUCTION

The safety of women while travelling, working late or walking alone remains a serious concern. Incidents of harassment, stalking and assault often occur in situations where the victim cannot make a phone call, unlock the phone and dial a number, or explain their location in time. Even when help is called, responders may not know the exact location, and the location keeps changing if the victim is in a moving vehicle.

Emergency numbers such as 112 and women's helpline 181 exist, and some apps provide SOS buttons, but many require multiple steps, work only when the screen is unlocked, or send only a single location without continuous tracking. People also have no easy way of knowing which routes are poorly lit or unsafe when planning a journey.

This project, SafeHer, is a mobile application with an optional wearable button. With one tap, a shake gesture, a voice keyword or the wearable button, the app sends a live location link and a short audio recording to trusted contacts and to a police control room dashboard. Tracking continues until the user marks herself safe. A safe-route feature suggests routes that pass through well-lit and crowded areas, and anonymised SOS data produces a heat map of unsafe zones for authorities.

LIMITATIONS

1. Victims often cannot call for help in time.
2. Emergency numbers need multiple steps to reach.
3. Single location messages become outdated in moving vehicles.
4. Unsafe areas are not visible when planning routes.

1.1 History of Domain

Personal alarms and pepper sprays have long been used for personal safety. With mobile phones, emergency calling became more accessible, and in India the single emergency number 112 was launched under the Emergency Response Support System (ERSS) in 2019, with a panic button feature mandated in new mobile phones since 2017.

Smartphone safety apps such as Himmat (Delhi Police), Kavalan SOS (Tamil Nadu Police) and Raksha have provided SOS features with location sharing. Research on wearable safety devices using GSM/GPS modules and on crime hotspot mapping using geospatial analysis has grown, supporting data-driven safety planning in smart cities.

1.2 Overview of the Project

The user registers with a phone number (OTP) and adds up to five trusted contacts. SOS can be triggered in four ways: pressing the large SOS button, shaking the phone three times (accelerometer), speaking a configured keyword (on-device keyword spotting), or pressing an ESP32-based Bluetooth wearable button. A short countdown allows cancellation of false alarms.

Once triggered, the app starts a foreground location service that uploads GPS coordinates every 5 seconds to Firebase, sends SMS (via Twilio) and push alerts to contacts with a live tracking link, records a 30-second audio clip and uploads it, and creates an incident on the control room dashboard with the nearest police station. Tracking continues until the user enters a PIN to mark herself safe. The safe-route module scores alternative routes from Google Maps by street lighting and crowd data (from open data and past SOS density) and recommends the safest reasonable route.

1.3 Motivation

In an emergency every second matters, and the person in danger may not be able to speak or operate the phone normally.

- **Motivation:** To make calling for help possible in one action, with continuous location sharing until the person is safe.
- **Outcome:** Faster response, accurate tracking of moving victims, deterrence, and data for improving public safety infrastructure.

1.4 Aim of the Project

The project is entitled as “**WOMEN SAFETY ALERT AND LIVE TRACKING SYSTEM**”. The aim of this project is to develop a women safety system consisting of a mobile application and optional wearable that sends instant SOS alerts with live tracking and audio to trusted contacts and the police, suggests safer routes, and produces unsafe-zone heat maps.

1.5 Objectives

- To provide multiple SOS triggers: button, shake, voice keyword and wearable.
- To send SMS and push alerts with a live tracking link.
- To continuously track location until the user marks safe.
- To record and upload audio evidence.
- To suggest safe routes and generate unsafe-zone heat maps.

1.6 Literature Review

Ministry of Home Affairs - ERSS (2019): The Emergency Response Support System provides a single emergency number 112 integrated with location-based dispatch of police, fire and ambulance services across Indian states.

DoT Panic Button Rules (2016): The Department of Telecommunications mandated a physical panic button and built-in GPS in mobile phones sold in India from 2017 and 2018 respectively.

Tamil Nadu Police - Kavalan SOS app (2019): The state police SOS application lets users send their location and an alert to the police control room and to contacts by pressing an SOS button or shaking the phone, showing the demand for one-touch safety tools.

Chainey, Tompson and Uhlig (2008): Compared hotspot mapping techniques for crime prediction and found kernel density estimation to be among the most accurate for identifying future crime locations.

Welsh and Farrington (2008): A systematic review showing that improved street lighting is associated with significant reductions in crime, supporting the use of lighting data in safe routing.

Firebase Realtime Database / Firestore: Provides real-time synchronisation of data to connected clients, suitable for continuously updating live location for viewers of a tracking link.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

In the existing system, the victim must call 112, 181 or a family member, explain the situation and describe the location. Some police apps send one SOS message with the location. There is no continuous tracking, no audio evidence and no safe-route guidance.

DISADVANTAGES

- Calling takes time and may not be possible.
- Single location becomes outdated.
- No evidence collection.
- Contacts and police are not alerted simultaneously.
- No information about unsafe areas.

2.2 Proposed System

The proposed SafeHer system provides one-action SOS through multiple triggers, simultaneous alerts to contacts and the police control room, continuous live tracking with a shareable link, automatic audio recording, a safe-route recommendation and a heat map of unsafe zones for authorities.

ADVANTAGES

- SOS without unlocking the phone (shake, voice, wearable).
- Live location until marked safe.
- Audio evidence automatically uploaded.
- Police dashboard with nearest station.
- Safe-route suggestions and hotspot analysis.

2.3 Project Schedule

The development of SafeHer follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the SafeHer system are listed below.

- **Registration:** Phone OTP login and profile.
- **Trusted contacts:** Add, edit and verify up to five contacts.
- **SOS triggers:** Button, shake, voice keyword and Bluetooth wearable.
- **Alerts:** SMS and push notification with live link.
- **Live tracking:** Upload location every 5 seconds until safe.
- **Audio recording:** Record and upload 30-second clips.
- **Control room:** Incident dashboard with map and nearest station.
- **Safe route and heat map:** Route scoring and unsafe zone visualisation.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Speed:** Alerts delivered within 10 seconds of trigger.
- **Reliability:** SMS fallback when internet is unavailable.
- **Battery:** Background service optimised; tracking only during SOS.

- **Privacy:** Location shared only during SOS; links expire after the incident.
- **Security:** Mark-safe requires PIN to prevent attacker cancelling.
- **Usability:** Large SOS button usable in panic.

3.3 User Requirements

- Trigger SOS quickly even without unlocking the phone
- Contacts can see live location on a map
- Police are notified automatically
- Find a safer route to travel
- Stop the alert only with a PIN

3.4 System Requirements

- Android app (Kotlin or Flutter)
- Firebase Auth, Firestore, Storage, Cloud Messaging
- Twilio SMS
- Google Maps API
- Optional ESP32 BLE wearable

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System : Intel i5 Processor 2.4 GHz or above

Hard Disk : 256 GB

RAM : 8 GB

Mobile : Android 9+ smartphone with GPS and accelerometer

Wearable (optional) : ESP32 with push button, vibration motor and battery

4.2 IoT Hardware Components

S.No	Component	Purpose
1	ESP32-C3 mini board	BLE communication with the phone
2	Tactile push button	Hidden SOS trigger (long press 2 s)
3	Vibration motor	Silent confirmation to the user
4	3.7 V LiPo battery 400 mAh + TP4056 charger	Power supply
5	3D printed pendant / keychain case	Wearable enclosure

4.3 Software Specification

Operating System : Android / Windows 10 for development

Mobile App : Kotlin (Android) or Flutter

Back End : Firebase Cloud Functions (Node.js)

Database : Firebase Firestore

Maps : Google Maps SDK and Directions API

SMS : Twilio

Wearable firmware : Arduino IDE (C++) for ESP32 BLE

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed SafeHer system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

Android provides location, accelerometer and audio APIs; Firebase provides real-time data sync and notifications; Twilio provides SMS; ESP32 supports BLE. All components are proven. The project is technically feasible.

ECONOMIC FEASIBILITY

The app is free to install and Firebase's free tier supports a pilot. The optional wearable costs about Rs. 600 to build. SMS costs are low. It is economically feasible.

OPERATIONAL FEASIBILITY

Users only need to add contacts once. In an emergency, one action triggers everything. The control room dashboard fits into existing police response workflows. It is operationally feasible.

5.2 About the Software

KOTLIN (ANDROID)

Kotlin is a modern, statically typed programming language that is the preferred language for Android development. It is concise, null-safe and fully interoperable with Java.

Android APIs such as FusedLocationProviderClient, SensorManager (accelerometer for shake detection) and SmsManager are directly available in Kotlin.

FLUTTER

Flutter is an open-source UI software development kit created by Google. It is used to build natively compiled applications for Android, iOS, web and desktop from a single code base written in the Dart language.

Flutter's hot reload feature and rich widget library speed up mobile development, and plugins are available for GPS location, sensors, maps and Firebase.

- Single code base for Android and iOS
- Fast development with hot reload
- Native performance

FIREBASE

Firebase is a Backend-as-a-Service platform from Google. It provides Cloud Firestore (a real-time NoSQL database), Authentication, Cloud Storage, Cloud Messaging (push notifications) and Cloud Functions.

Firestore pushes data changes to all connected clients instantly, which is ideal for live tracking and real-time notifications without writing a separate socket server.

- Real-time database synchronisation
- Phone OTP authentication
- Free Spark plan for prototypes

GOOGLE MAPS PLATFORM

Google Maps Platform provides APIs for embedding maps, geocoding addresses, calculating distances and generating driving or walking directions.

The Directions and Distance Matrix APIs are used to find the nearest volunteer or police station and to plan routes.

TWILIO SMS / TELEGRAM BOT API

Twilio is a cloud communications platform that offers APIs for sending SMS, making voice calls and sending WhatsApp messages from code. The Telegram Bot API is

a free HTTP-based interface for sending messages to Telegram users and groups.

These services let the system deliver alerts directly to users' phones within seconds, even if they are not using the application at that time.

ESP32 MICROCONTROLLER

The ESP32 is a low-cost, low-power system-on-chip microcontroller from Espressif Systems with integrated Wi-Fi and dual-mode Bluetooth. It has a dual-core Tensilica Xtensa LX6 processor running up to 240 MHz, 520 KB SRAM, and many GPIO, ADC, I2C, SPI and UART pins.

The ESP32 can be programmed using the Arduino IDE in C/C++ or with MicroPython. Its built-in Wi-Fi lets sensor readings be sent directly to a cloud server using MQTT or HTTP without any additional module.

- Built-in Wi-Fi and Bluetooth
- 12-bit ADC for analog sensors
- Deep-sleep mode for battery operation
- Costs less than Rs. 500

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of SafeHer is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

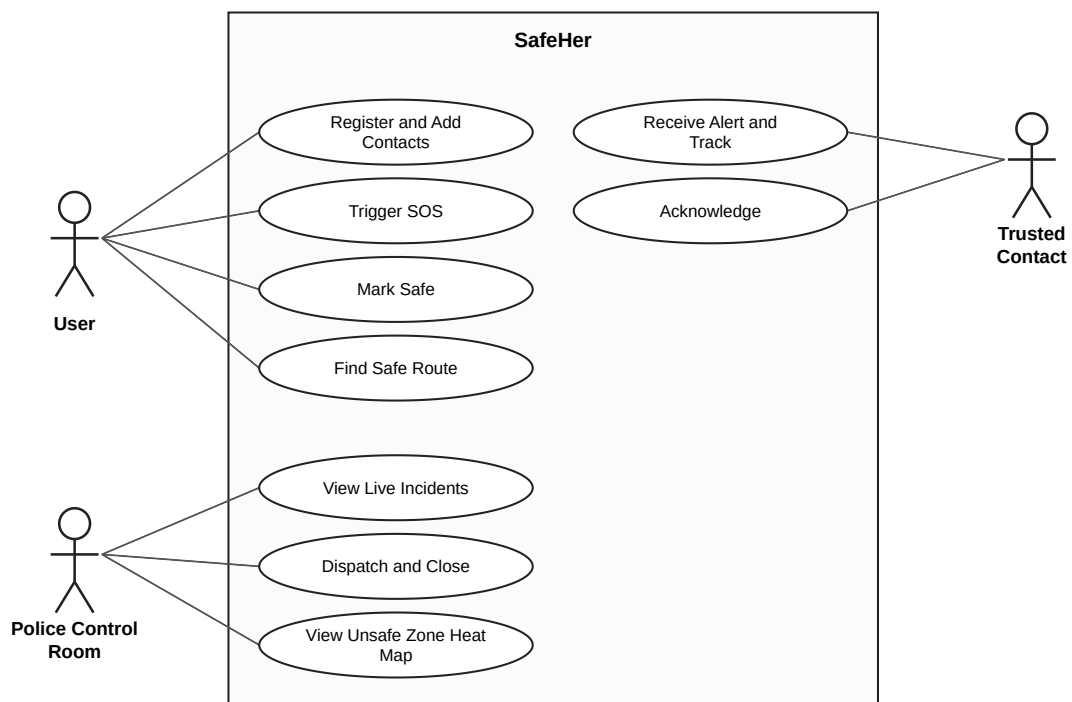


Fig 1: Use case diagram of SafeHer

6.2 System Architecture

SafeHer consists of the mobile app with background services, an optional BLE wearable, Firebase cloud services and a web dashboard for the police control room.

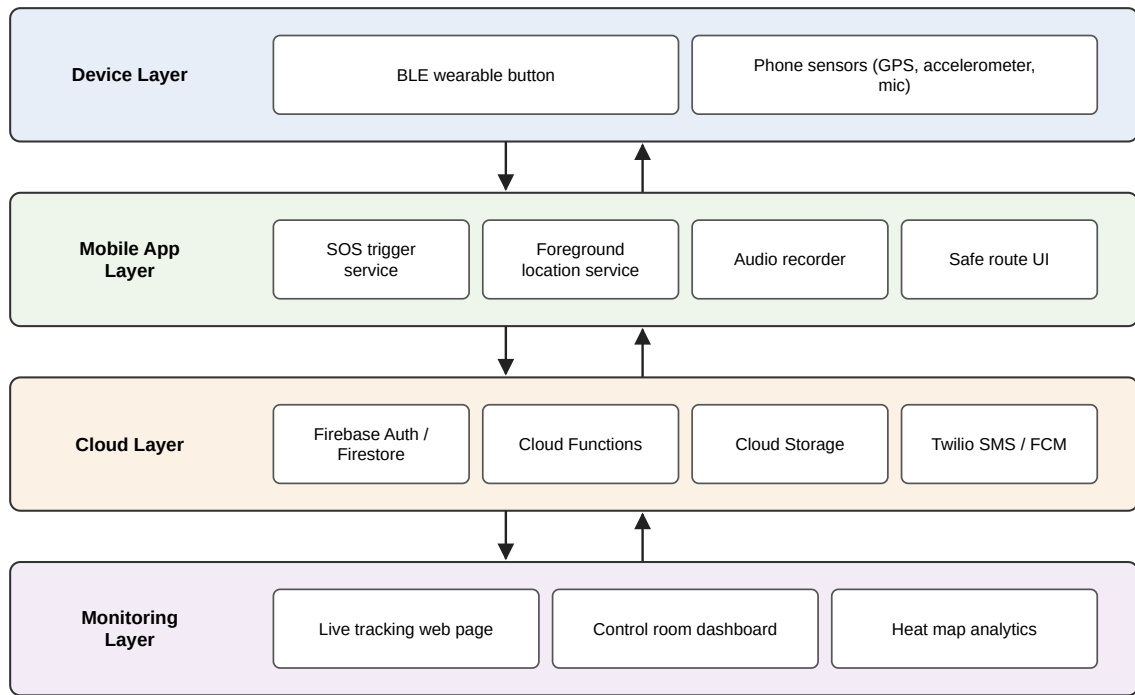


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

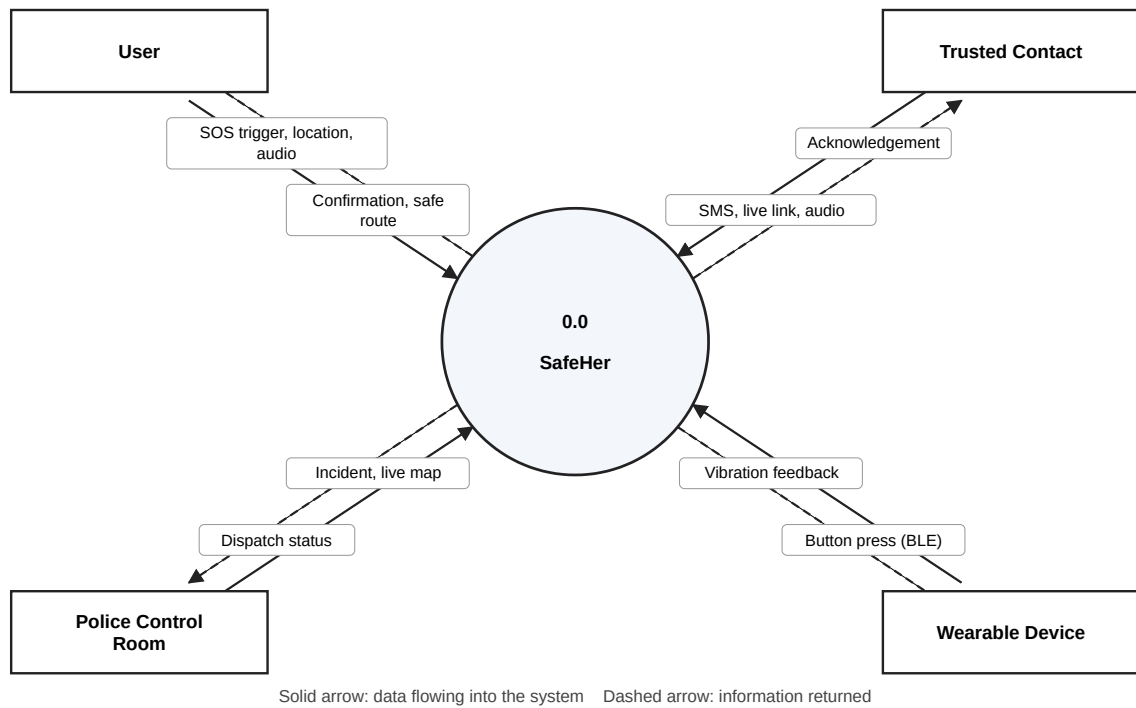


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

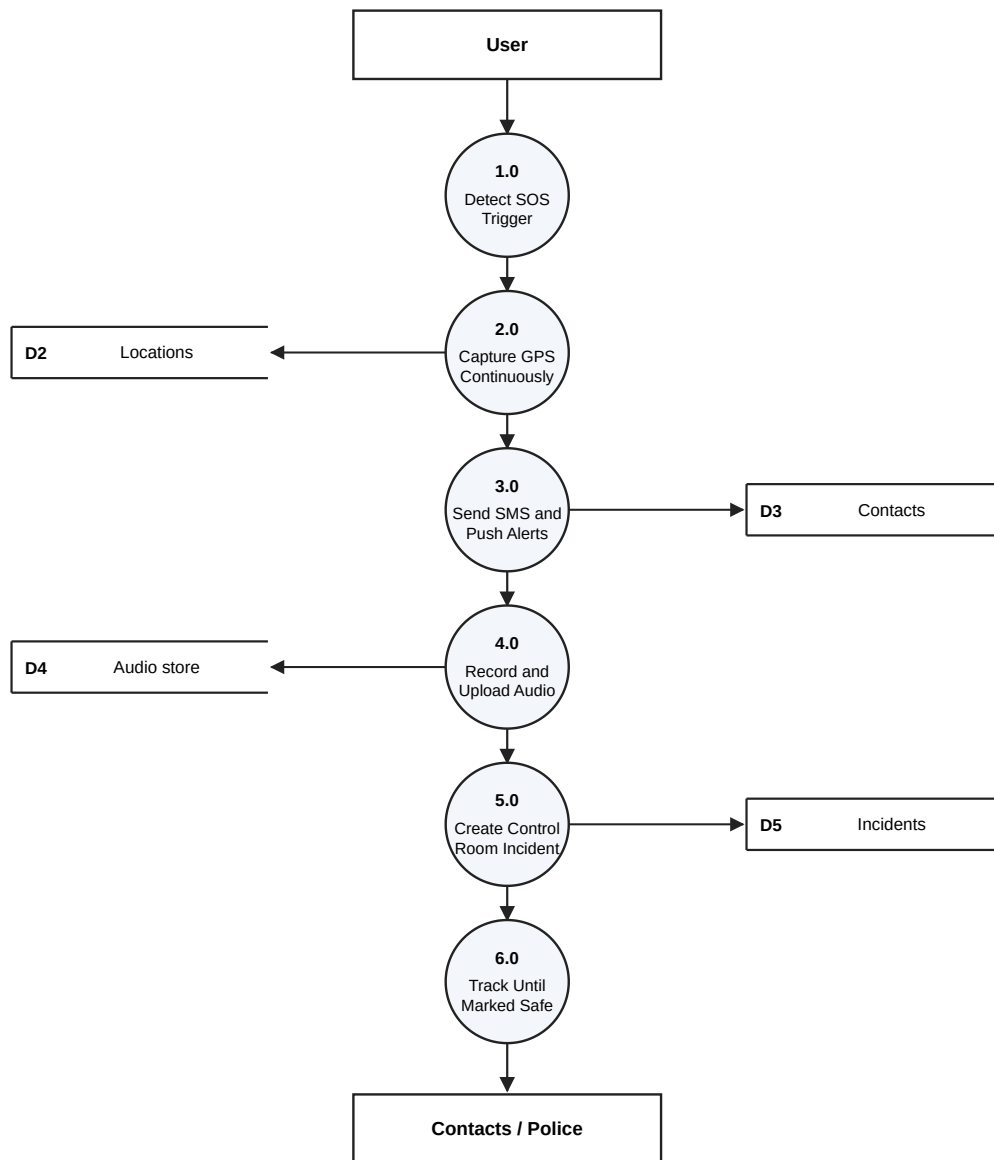


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

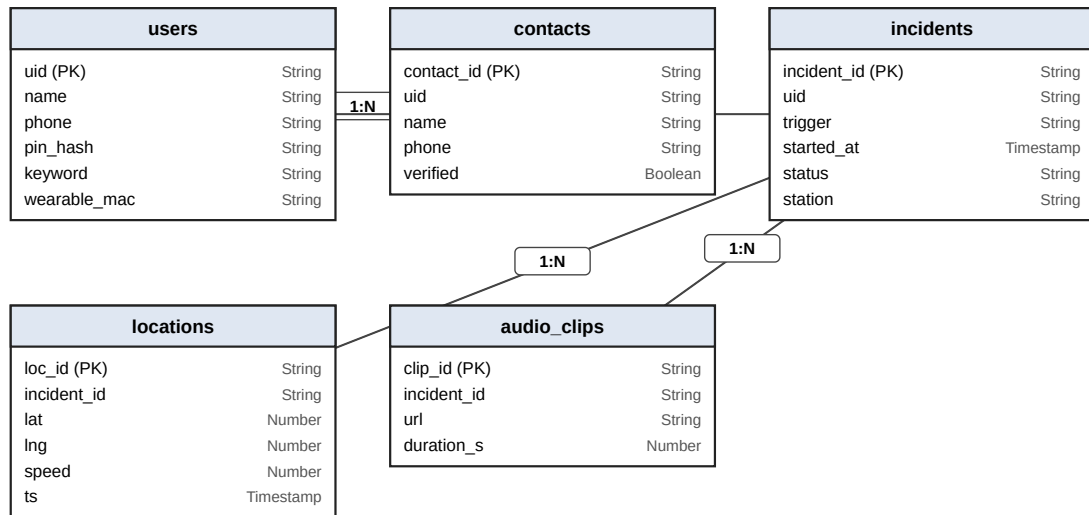


Fig 5: Entity relationship diagram

6.5 Table Design

USERS

FIELD NAME	DATA TYPE	DESCRIPTION
uid	String	Primary key
name	String	User name
phone	String	Phone (OTP verified)
pin_hash	String	Hashed safe PIN
keyword	String	Voice keyword
wearable_mac	String	Paired wearable

CONTACTS

FIELD NAME	DATA TYPE	DESCRIPTION
contact_id	String	Primary key
uid	String	Foreign key to users
name	String	Contact name
phone	String	Phone
verified	Boolean	Confirmed by SMS

INCIDENTS

FIELD NAME	DATA TYPE	DESCRIPTION
incident_id	String	Primary key
uid	String	Foreign key to users
trigger	String	button / shake / voice / wearable
started_at	Timestamp	Start time
status	String	active / safe / closed
station	String	Nearest police station

LOCATIONS

FIELD NAME	DATA TYPE	DESCRIPTION
loc_id	String	Primary key
incident_id	String	Foreign key to incidents
lat	Number	Latitude
lng	Number	Longitude
speed	Number	Speed m/s
ts	Timestamp	Time

AUDIO_CLIPS

FIELD NAME	DATA TYPE	DESCRIPTION
clip_id	String	Primary key
incident_id	String	Foreign key to incidents
url	String	Storage URL
duration_s	Number	Duration

6.6 Algorithm Design

The SOS workflow and safe route scoring algorithm are described below.

1. **User sets trusted contacts** – Contacts are verified via SMS confirmation.

2. **SOS triggered by button, shake or voice keyword** – Shake: acceleration magnitude above 2.5 g three times within 2 seconds; voice: on-device keyword spotting; wearable: BLE notification on long press.
3. **Capture GPS location continuously** – FusedLocationProvider with high accuracy every 5 seconds in a foreground service.
4. **Send SMS and app alert with live link** – Cloud Function sends Twilio SMS and FCM push with `https://safeher.app/t/<incident_id>`.
5. **Record short audio and upload** – MediaRecorder records 30-second AAC clips uploaded to Cloud Storage, repeated while active.
6. **Notify nearest control room via API** – Nearest police station is computed by haversine distance; an incident appears on that station's dashboard.
7. **Keep tracking until user marks safe** – Mark-safe requires PIN; a duress PIN silently continues the alert.
8. **Report heat map of unsafe zones for authorities** – Kernel density estimation over anonymised incident locations by time band.

PSEUDO CODE

```
ON SENSOR_CHANGED(ax, ay, az):  
  g <- SQRT(ax^2 + ay^2 + az^2) / 9.81  
  IF g > 2.5 THEN shakes.append(now); shakes <- last 2 s  
  IF |shakes| >= 3 THEN START_SOS("shake")  
  
START_SOS(trigger):  
  VIBRATE; COUNTDOWN 5 s (cancel allowed)  
  id <- CREATE_INCIDENT(trigger, LOCATION())  
  START_FOREGROUND_TRACKING(id, every 5 s)  
  START_AUDIO_LOOP(id, 30 s clips)  
  // Cloud Function on incident create: SMS + push to contacts, notify nearest  
  station  
  
MARK_SAFE(pin):  
  IF HASH(pin) = user.pin_hash THEN STOP_ALL; incident.status <- safe  
  ELSE IF HASH(pin) = user.duress_hash THEN SHOW "Cancelled" BUT CONTINUE; FLAG  
  duress
```

6.7 Modules

The modules are grouped according to the User, Trusted Contact and Police Control Room.

MODULES

USER

1. REGISTER AND ADD CONTACTS
2. TRIGGER SOS
3. MARK SAFE
4. FIND SAFE ROUTE

TRUSTED CONTACT

1. RECEIVE ALERT AND TRACK
2. ACKNOWLEDGE

POLICE CONTROL ROOM

1. VIEW LIVE INCIDENTS
2. DISPATCH AND CLOSE
3. VIEW UNSAFE ZONE HEAT MAP

USER

REGISTER AND ADD CONTACTS

The user registers with phone OTP and adds trusted contacts who receive an SMS to confirm.

TRIGGER SOS

SOS is triggered by button, three shakes, voice keyword or wearable button. A 5-second countdown with vibration allows cancellation of accidental triggers.

MARK SAFE

Tracking and alerts stop only after the user enters her PIN; a fake PIN can be configured to appear to cancel while silently continuing the alert.

FIND SAFE ROUTE

The user enters a destination and sees route options with a safety score.

TRUSTED CONTACT

RECEIVE ALERT AND TRACK

Contacts receive an SMS and push notification with a link to a live map that updates every 5 seconds, plus the audio clip.

ACKNOWLEDGE

Contacts tap 'I am responding', which is shown to the user and the control room.

POLICE CONTROL ROOM

VIEW LIVE INCIDENTS

Officers see active SOS incidents on a map with the user's live location, nearest police station and patrol vehicle.

DISPATCH AND CLOSE

Officers record dispatch details and close the incident.

VIEW UNSAFE ZONE HEAT MAP

Anonymised incident locations are shown as a heat map by time of day.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

The SafeHer Android app was implemented in Kotlin with a foreground service for SOS detection and tracking. Firebase provides authentication, real-time location storage and push notifications; Cloud Functions send SMS through Twilio and create incidents for the nearest police station. The live tracking page and control room dashboard are web pages using the Google Maps JavaScript API that listen to Firestore updates.

7.2 Implementation

The optional wearable was built with an ESP32-C3 that advertises a BLE service; a long press of 2 seconds sends a notification to the paired phone, which starts the SOS. The safe-route module requests alternative routes from the Directions API and scores them using a street-light density layer (from municipal open data where available) and historical SOS density.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.

- To validate the software against its requirements.
- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	SOS button	Tap SOS	Countdown then alerts sent	Pass
TC02	Shake trigger	Shake phone 3 times	SOS started	Pass
TC03	False alarm cancel	Cancel during countdown	No alerts sent	Pass
TC04	Live tracking	User walking	Contact map updates every 5 s	Pass
TC05	No internet	Mobile data off	SMS with last known location sent	Pass
TC06	Wearable	Long press wearable button	SOS started on phone	Pass
TC07	Mark safe with PIN	Correct PIN	Tracking stopped	Pass
TC08	Duress PIN	Duress PIN entered	Appears cancelled; alert continues	Pass

PERFORMANCE EVALUATION

Measured performance during field trials (30 trials):

Measure	Value
Average time from trigger to SMS received	6.8 s
Average time to push notification	2.4 s
Location accuracy (outdoors)	4 - 8 m
Shake detection true positive rate	96.7%
Accidental shake triggers (per day, normal use)	0.1 (cancelled by countdown)

7.4 Source Code

SosService.kt (Android foreground service)

```
class SosService : Service(), SensorEventListener {
    private lateinit var fused: FusedLocationProviderClient
    private val db = Firebase.firestore
    private val shakes = ArrayDeque<Long>()
    private var incidentId: String? = null
    private var recorder: MediaRecorder? = null

    override fun onCreate() {
        super.onCreate()
        startForeground(1, buildNotification("SafeHer is protecting you"))
        fused = LocationServices.getFusedLocationProviderClient(this)
        val sm = getSystemService(SENSOR_SERVICE) as SensorManager
        sm.registerListener(this,
sm.getDefaultSensor(Sensor.TYPE_ACCELEROMETER), SensorManager.SENSOR_DELAY_GAME)
    }

    override fun onSensorChanged(e: SensorEvent) {
        val g = sqrt(e.values[0].pow(2) + e.values[1].pow(2) +
e.values[2].pow(2)) / 9.81f
        val now = System.currentTimeMillis()
        if (g > 2.5f && (shakes.isEmpty() || now - shakes.last() > 250))
shakes.addLast(now)
        while (shakes.isNotEmpty() && now - shakes.first() > 2000)
shakes.removeFirst()
        if (shakes.size >= 3 && incidentId == null) { shakes.clear();
startSos("shake") }
    }

    fun startSos(trigger: String) {
        val uid = Firebase.auth.currentUser!!.uid
        val ref = db.collection("incidents").document()
        incidentId = ref.id
        ref.set(mapOf("uid" to uid, "trigger" to trigger, "status" to "active",
        "started_at" to FieldValue.serverTimestamp()))
        val req = LocationRequest.Builder(Priority.PRIORITY_HIGH_ACCURACY,
5000).build()
        fused.requestLocationUpdates(req, locationCallback,
Looper.getMainLooper())
        recordAudioClip()
    }

    private val locationCallback = object : LocationCallback() {
        override fun onLocationResult(r: LocationResult) {
            val l = r.lastLocation ?: return
            val id = incidentId ?: return
            db.collection("incidents").document(id).collection("locations")
                .add(mapOf("lat" to l.latitude, "lng" to l.longitude, "speed" to
l.speed,
                "ts" to FieldValue.serverTimestamp()))
            db.collection("incidents").document(id).update("last",
GeoPoint(l.latitude, l.longitude))
        }
    }

    private fun recordAudioClip() {
        val file = File(cacheDir, "sos_${System.currentTimeMillis()}.m4a")
    }
}
```

```

        recorder = MediaRecorder(this).apply {
            setAudioSource(MediaRecorder.AudioSource.MIC);
setOutputFormat(MediaRecorder.OutputFormat.MPEG_4)
            setAudioEncoder(MediaRecorder.AudioEncoder.AAC);
setOutputFile(file.absolutePath)
            setMaxDuration(30_000)
            setOnInfoListener { _, what, _ ->
                if (what ==
MediaRecorder.MEDIA_RECORDER_INFO_MAX_DURATION_REACHED) {
                    stop(); release(); upload(file); if (incidentId != null)
recordAudioClip()
                }
            }
            prepare(); start()
        }
    }

    private fun upload(f: File) {
        val id = incidentId ?: return

        Firebase.storage.reference.child("audio/$id/${f.name}").putFile(Uri.fromFile(f))
    }

    fun markSafe() {
        fused.removeLocationUpdates(locationCallback)
        incidentId?.let {
            db.collection("incidents").document(it).update("status", "safe")
            incidentId = null
        }
    }

    override fun onAccuracyChanged(s: Sensor?, a: Int) {}
    override fun onBind(i: Intent?) = null
}

```

functions/sos.js (Cloud Function)

```

const { onDocumentCreated } = require("firebase-functions/v2/firestore");
const admin = require("firebase-admin");
const twilio = require("twilio")(process.env.TW_SID, process.env.TW_TOKEN);
admin.initializeApp();
const db = admin.firestore();

const hav = (a, b) => {
    const R = 6371, r = x => x * Math.PI / 180;
    const d = Math.sin(r(b.lat) - a.lat) / 2) ** 2 +
        Math.cos(r(a.lat)) * Math.cos(r(b.lat)) * Math.sin(r(b.lng) - a.lng)
    / 2) ** 2;
    return 2 * R * Math.asin(Math.sqrt(d));
};

exports.onSos = onDocumentCreated("incidents/{id}", async ev => {
    const inc = ev.data.data(), id = ev.params.id;
    const user = (await db.doc(`users/${inc.uid}`).get()).data();
    const link = `https://safeher.app/t/${id}`;
    const text = `SOS from ${user.name}! She may be in danger. Live location:
    ${link}`;
    const contacts = await db.collection("contacts").where("uid", "=",
    inc.uid).where("verified", "=", true).get();
    await Promise.all(contacts.docs.map(c =>
        twilio.messages.create({ to: c.get("phone"), from: process.env.TW_FROM,
        body: text })));
});

```

```

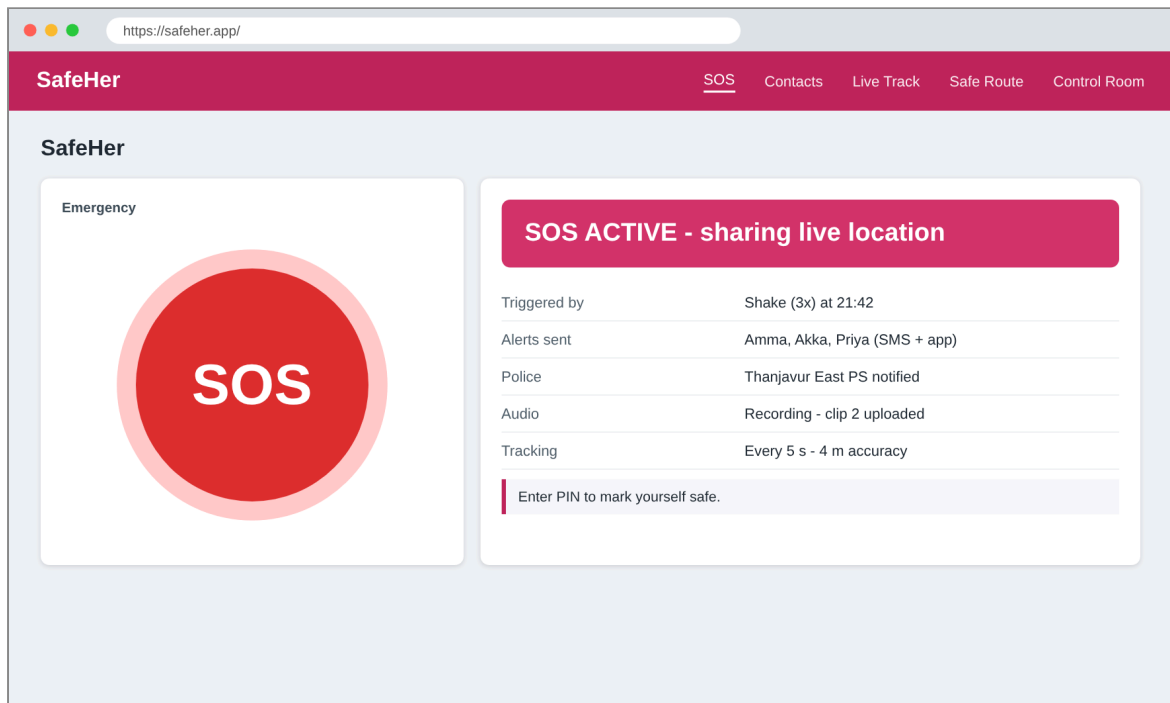
const stations = (await db.collection("stations").get()).docs.map(d => ({ id:
d.id, ...d.data() }));
const here = { lat: inc.last?.latitude ?? user.home_lat, lng:
inc.last?.longitude ?? user.home_lng };
const nearest = stations.sort((a, b) => hav(here, a) - hav(here, b))[0];
await ev.data.ref.update({ station: nearest.id, station_name: nearest.name });
await admin.messaging().send({ topic: `station_${nearest.id}`,
notification: { title: "New SOS incident", body: `${user.name} - ${link}` }
});
});

```

7.5 Screen Layout

SCREENSHOTS:

SOS SCREEN



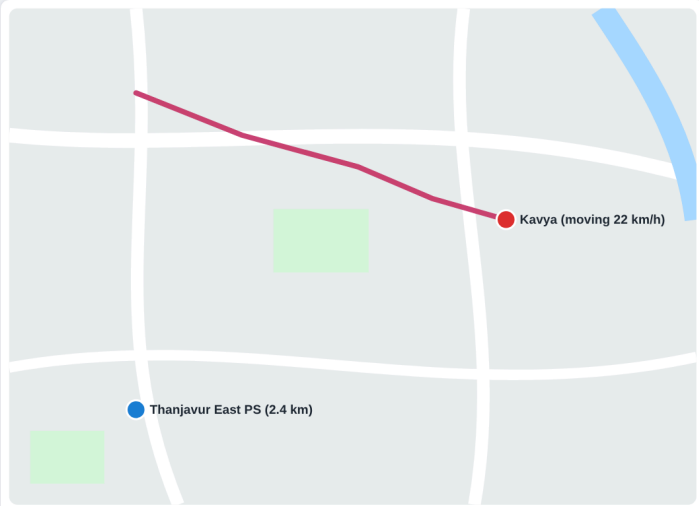
LIVE TRACKING (CONTACT VIEW)

https://safeher.app/

SafeHer

SOSContactsLive TrackSafe RouteControl Room

Live location of Kavya - updated 3 s ago



Status	SOS ACTIVE
Started	21:42
Last update	21:51:08
Speed	22 km/h (vehicle)
Audio	▶ clip 1 ▶ clip 2
Responding	Akka, Patrol vehicle TN-49-G-1123

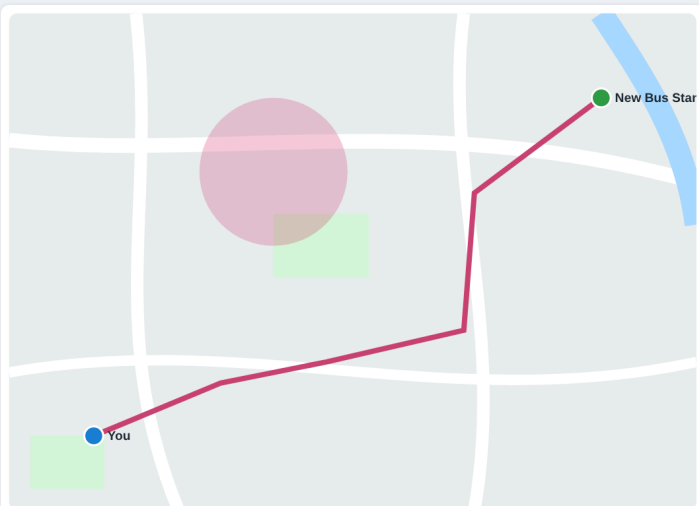
SAFE ROUTE SUGGESTION

https://safeher.app/

SafeHer

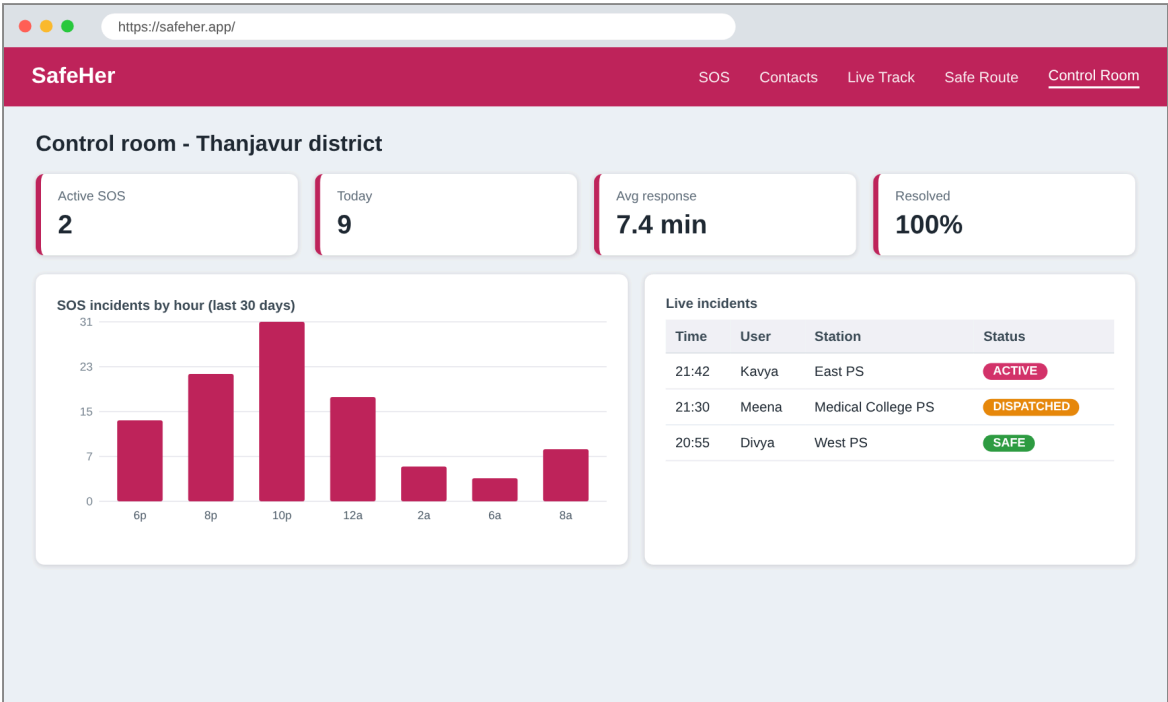
SOSContactsLive TrackSafe RouteControl Room

Safe route to Bus Stand



Recommended	Via Gandhiji Road
Safety score	8.6 / 10 (well lit, busy)
Time	14 min walk
Avoided	Canal road - poor lighting, 3 past SOS
Shortest route	11 min, score 4.2

POLICE CONTROL ROOM



CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The Women Safety Alert and Live Tracking System enables a person in danger to call for help with a single action - a button tap, a shake, a voice keyword or a hidden wearable button. Alerts with a live tracking link reached contacts in under 7 seconds on average, and tracking continued until the user confirmed her safety with a PIN. Audio evidence is captured automatically.

The police control room dashboard and unsafe-zone heat maps support faster response and better planning of patrols and street lighting. The system offers a practical, low-cost improvement in personal safety.

8.2 Future Enhancement

- Integration with the national 112 ERSS API.
- Smartwatch app and fall/struggle detection using motion sensors.
- Offline mesh alerts using Bluetooth to nearby SafeHer users.
- Automatic video recording with cloud backup.
- Community volunteer responders within 500 m.

CHAPTER IX

BIBLIOGRAPHY

1. Ministry of Home Affairs, Government of India, "Emergency Response Support System (ERSS) - 112", 2019.
2. Department of Telecommunications, "Panic Button and Global Positioning System in Mobile Phone Handsets Rules", 2016.
3. Chainey, S., Tompson, L., and Uhlig, S., "The Utility of Hotspot Mapping for Predicting Spatial Patterns of Crime", Security Journal, Vol. 21, pp. 4-28, 2008.
4. Welsh, B. C., and Farrington, D. P., "Effects of Improved Street Lighting on Crime", Campbell Systematic Reviews, Vol. 4, No. 1, 2008.
5. Google, Android Developers Documentation - Location and Sensors, <https://developer.android.com>.

WEBSITE REFERENCE

1. <https://developer.android.com>
2. <https://firebase.google.com/docs>
3. <https://www.twilio.com/docs/sms>
4. <https://112.gov.in>